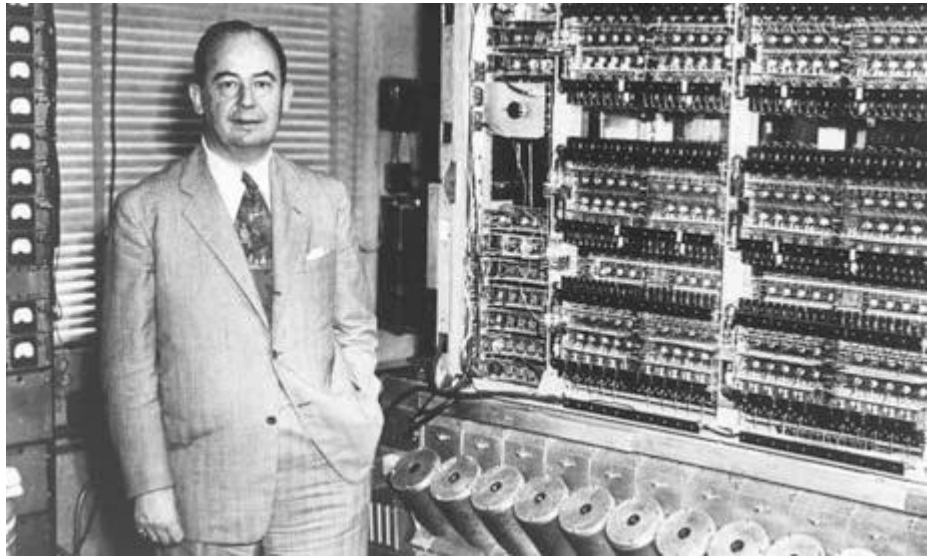
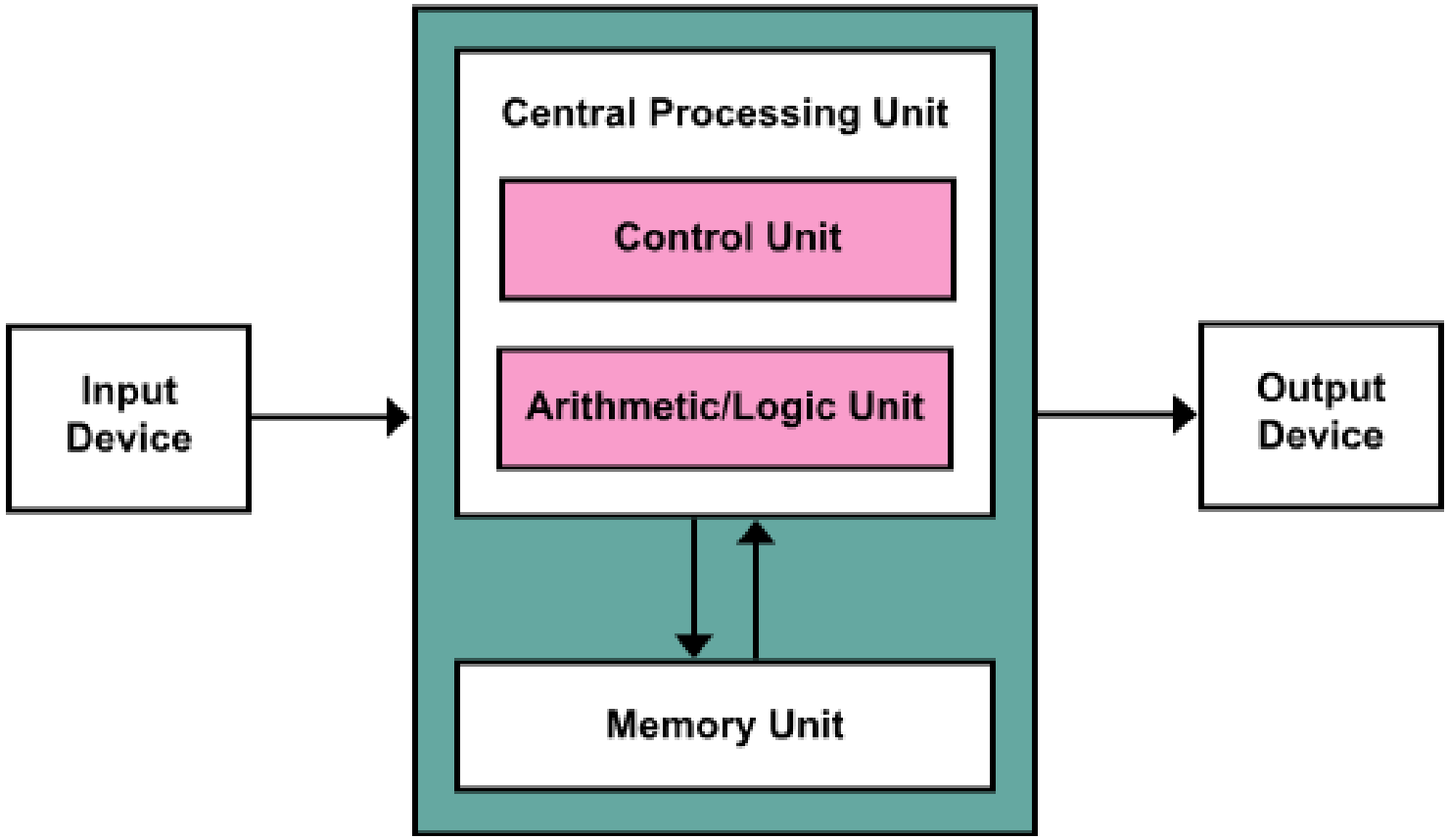
Operating Systems

Dmitry Zaitsev

Lecture 20: Computer architecture overview OS interface with hardware

Von Neumann architecture of computer





Central Processing Unit (CPU)

- Registers – quick memory – general purpose and specialized: accumulator, base register, index register, etc
- Arithmetic-logic unit (ALU)
- Control unit (CU)
- Program counter (PC)
- Stack pointer (SP)
- Processor-state word (PSW)
- Flags of PSW

Memory Unit

- An indexed array of bytes (octets – 8 bits)
- A bite is referenced to by its address – index in the array
- Maximal size is defined by the address bus width
- Memory word – a few bytes, 4 or 8; its width is defined by the data bus width
- Word is extracted in a single memory access
- Memory is universal – stores instructions (programs) and data; binary encoded

How CPU works

- Extracts the current instruction pointed by PC from memory
- Recognizes code of operation (COP)
- Fetches operands from registers and memory
- Starts ALU to implement the operation
- Stores result to a register (memory)
- Increments PC

- Some operations do not involve ALU: jumps – just change PC; routine call – store return address in stack and jump; etc

Instruction set

- Memory movement: processor registers - RAM
- Arithmetic operations: addition, subtraction, multiplication, division, etc
- Logic operations: conjunction, disjunction, negation, etc
- Shifts: left, right, cyclic, etc
- Comparisons
- Conditional and unconditional jumps; condition based on flags
- Routine call and return
- Special instructions

Addressing modes

- Register
- Direct memory – memory address inside instruction
- Based – memory offset inside instruction with respect to base address stored in special register
- Immediate – operand (constant) inside the instruction
- Indirect memory – a register contains memory address
- Indexed memory – registers contain memory address and index

Interaction with I/O devices

- Registers (ports) of I/O devices (controllers)
- Interrupts of I/O devices
- Interrupt control – enable or disable

- Basic communication technique:

✓Registers of devices are mapped on the memory address space – use the same commands as for the memory transfer

✓Register of devices represent a separate address space – use special operations for reading and writing registers

X86 architecture I/O

- Ports are used for communication between the CPU and I/O devices
- Separate address space from memory
- Each port has a unique number (0–65535)
- IN (input) – read from port
- OUT (output) – write to port
- Example:

```
IN AL, 60h    ; Read from keyboard port
OUT 64h, AL   ; Send command to control port
```

Program Status Word (PSW)

- Flags – status of CPU operations: Carry, Zero, Sign, Overflow
- Enabled-disabled interrupts
- I/O Privilege Level (bits 12-13): 0 (Kernel) – 3 (User)
- Also Code Segment Register – Current Privilege Level: 0 (Kernel) – 3 (User)

Assembly X86 program for branching in C

- if(x>y) z=x; else z=y;

```
        movl  -12(%rbp), %eax   # x->bx
        cmpl  -8(%rbp), %eax    # cmp x y
        jle   L2
        movl  -12(%rbp), %eax   # x->ax
        movl  %eax, -4(%rbp)    # ax->z
        jmp   L3
L2:
        movl  -8(%rbp), %eax    # y->ax
        movl  %eax, -4(%rbp)    # ax->z
L3:
```

Assembly X86 program for loop in C

- while(x>y) z=x+y;

```
L6:
        movl  -12(%rbp), %eax   # x->ax
        cmpl  -8(%rbp), %eax    # cmp x, y
        jle   .L7
        movl  -12(%rbp), %edx   # x->dx
        movl  -8(%rbp), %eax    # y->ax
        addl  %edx, %eax        # x+y->ax
        movl  %eax, -4(%rbp)    # ax->z
        jmp L6:
L7:
```

Overview of computer hardware design

- Theoretical foundations:

✓Binary logic and minimization of Boolean function – combinatorial logic

✓State machine (finite automata) theory and minimization of automata – sequential logic

- Hardware Design Language – a specialized computer language used to describe the structure and behavior of electronic circuits.
- HDL provides a formal description of an electronic circuit for the automated analysis and simulation of the circuit, the synthesis of an HDL description into a specification of physical electronic components and how they are connected together, which can then be placed and routed to produce the set of masks used to create an integrated circuit.
- Verilog originated in 1983 at Gateway Design Automation

Logic gates

Name	NOT	AND	NAND	OR	NOR	XOR	XNOR
Alg. Expr.	$\overline{A}$	AB	$\overline{AB}$	$A+B$	$\overline{A+B}$	$A \oplus B$	$\overline{A \oplus B}$
Symbol							
Truth Table	A | X 0 | **1** 1 | **0**	B A | X 0 0 | **0** 0 1 | **0** 1 0 | **0** 1 1 | **1**	B A | X 0 0 | **1** 0 1 | **1** 1 0 | **1** 1 1 | **0**	B A | X 0 0 | **0** 0 1 | **1** 1 0 | **1** 1 1 | **1**	B A | X 0 0 | **1** 0 1 | **0** 1 0 | **0** 1 1 | **0**	B A | X 0 0 | **0** 0 1 | **1** 1 0 | **1** 1 1 | **0**	B A | X 0 0 | **1** 0 1 | **0** 1 0 | **0** 1 1 | **1**

Verilog

- hardware description language (HDL)
- design and verification of digital circuits
- syntax similar to C
- standardized as IEEE 1364
- tools: ModelSim (siemens), VeriLogger, Verilator etc

Defining a Verilog Module

- Keyword *module* to begin the definition
- Identifier that is the name of the module
- Optional list of parameters
- Optional list of ports (to be addressed more deeply in a future article)
- Module item
- Keyword *endmodule* to end the definition

Full 1 bit adder

- One bit adder
- Input:
  - bit x
  - bit y
  - input carry c_in
- Output (s=x+y+c_in):
  - bit s – sum
  - bit c_out – output carry

```
   1   c_in
+
   1   x
+
   1   y
----
  11
  cs
```

Full adder design

x	y	c_in	c_out	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

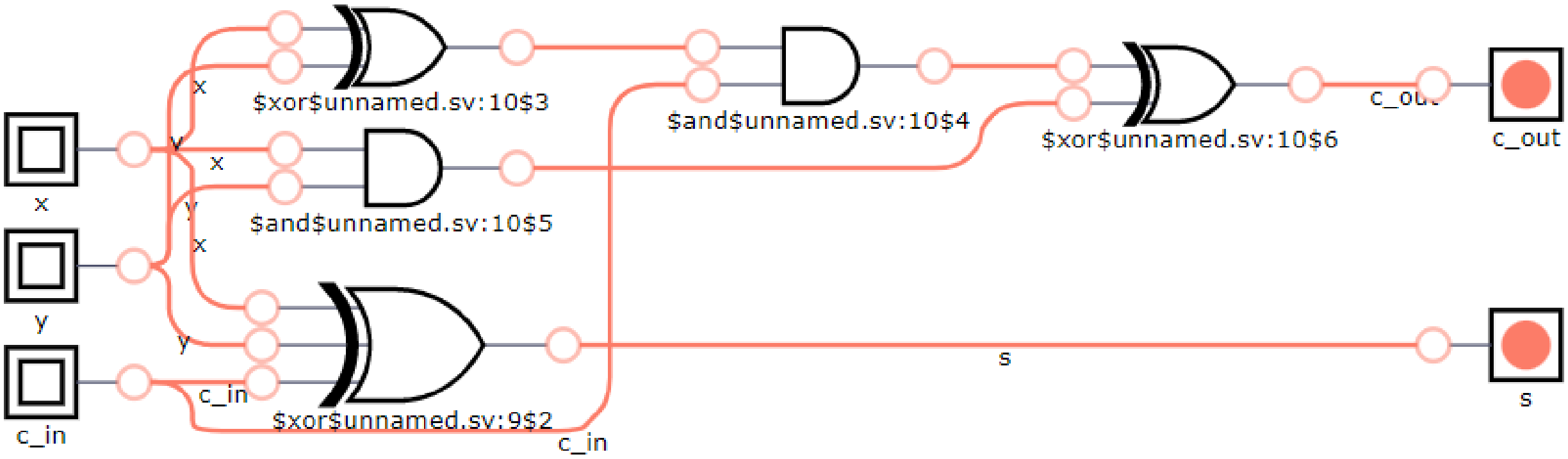
$$s = (x \oplus y) \oplus c_in$$

$$c_out = ((x \oplus y) \wedge c_in) \oplus (x \wedge y)$$

```
module fulladder1
( input x,
  input y,
  input c_in,

  output s,
  output c_out  );

  assign s =  (x^y) ^ c_in;
  assign c_out = ((x^y) & c_in) ^ (x&y);

endmodule
```

Full adder module

Verilog online: http://digitaljs.tilk.eu



Full adder scheme

Hierarchical design

- specify a bus as an array of wires
- specify components
- use components
- connect input and output wires of components

Four bits sequential adder

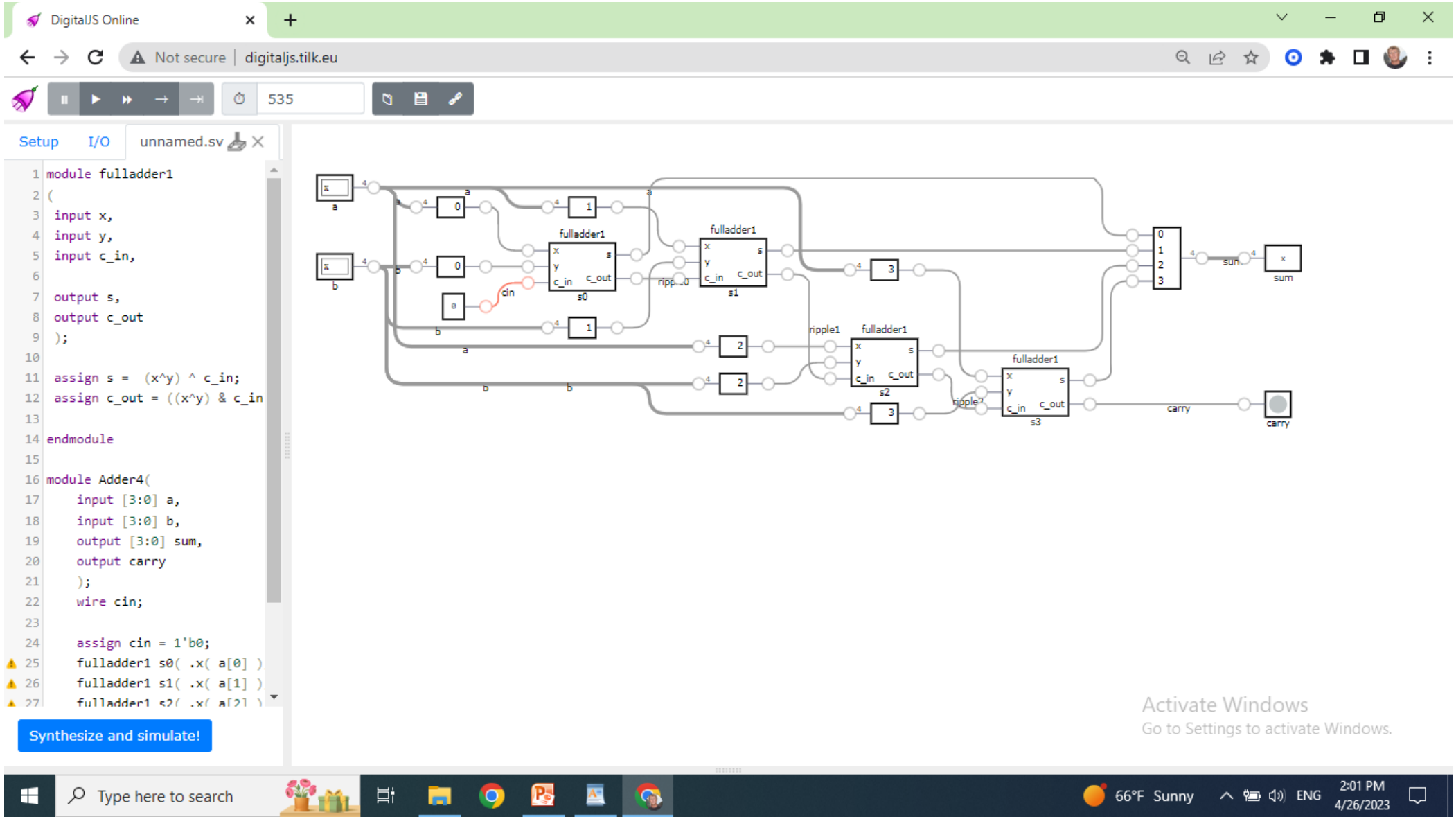
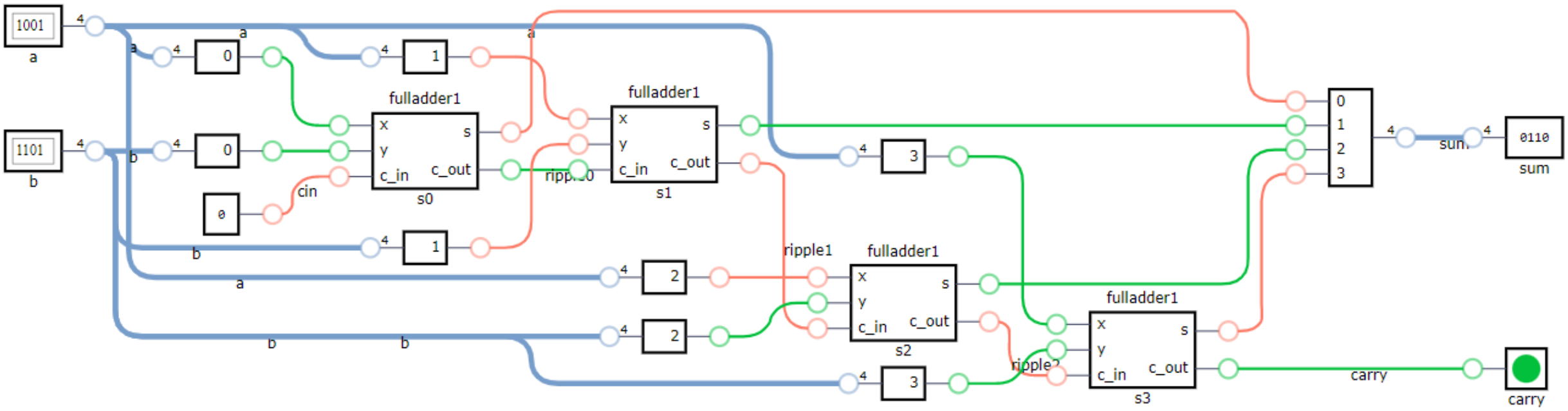
- idea: connect 4 full adders sequentially
- c_out of an adder to c_in of the next adder
- c_in of the first feed with 0
- use c_out of the last as the resulting carry
- example:

```
  1001
  1101
-------
 10110
```

Four bits sequential adder module

```
module Adder4(
   input [3:0] a,
   input [3:0] b,
   output [3:0] sum,
   output carry
   );
            wire cin;

            assign cin = 1'b0;
   fulladder1 s0( .x( a[0] ), .y( b[0]), .c_in( cin ), .s( sum[0]), .c_out( ripple0 ) );
   fulladder1 s1( .x( a[1] ), .y( b[1]), .c_in( ripple0 ), .s( sum[1]), .c_out( ripple1 ) );
   fulladder1 s2( .x( a[2] ), .y( b[2]), .c_in( ripple1 ), .s( sum[2]), .c_out( ripple2 ) );
   fulladder1 s3( .x( a[3] ), .y( b[3]), .c_in( ripple2 ), .s( sum[3]), .c_out( carry ) );

endmodule
```

Automatic layout



Manual testing